image_basics_teacher

March 4, 2026

1 Week 1: Image Basics & OpenCV

Computer Vision Fundamentals — Temasek Poly IIT AAI

(Teacher copy — full solutions and explanations. Use the student copy for in-class fill-in.)

This notebook introduces what an image is in the computer, how it is stored in memory, and how we work with images using OpenCV and NumPy.

1.1 2.1 Introduction to Computer Vision

Computer Vision (CV) is the field of teaching machines to "see"—to understand and interpret images and video.

1.1.1 History and applications

- **Security**: Face recognition, surveillance, access control
- **Manufacturing**: Quality inspection, defect detection, robotics
- **Agriculture**: Crop health, yield estimation, autonomous harvesters
- **Healthcare**: Medical imaging (X-rays, MRI), diagnosis support
- **Transport**: Self-driving cars, traffic monitoring, license plate recognition
- **Entertainment**: Augmented reality, photo filters, gaming

1.1.2 Traditional AI vs ML/DL

- **Traditional / classical CV**: Rule-based methods (edges, corners, thresholds). Fast, interpretable, works well when the problem is well-defined (e.g. barcode reading).
- **Machine Learning / Deep Learning**: Learn patterns from data. Better for complex, varied scenes (e.g. recognising any object).
- **In practice**: We often combine both—classical CV for preprocessing (e.g. finding regions), and ML/DL for recognition.

1.2 2.2 Image Data Representation

An image on disk (e.g. JPEG, PNG) is stored in a compressed format. When we **load** it into memory with OpenCV, it becomes a **NumPy array** of numbers.

1.2.1 Key ideas

- **Channels**: Colour images usually have 3 channels—Red, Green, Blue (RGB). Each pixel has 3 numbers.
- **Bit depth**: How many bits per channel. **8-bit** (one byte) per channel is very common → values from **0 to 255** (2 = 256 levels).
- **Grayscale**: One channel only. 0 = black, 255 = white.
- **RGB vs RGBA**: RGBA adds an **alpha** channel for transparency (e.g. PNG with transparency).
- **OpenCV convention**: When you read an image with `cv2.imread()`, the order of channels is **BGR** (Blue, Green, Red), not RGB. This matters when displaying with Matplotlib (which expects RGB) or when you index channels.

1.2.2 Connecting bits/bytes to the screen

- 8 bits = 1 byte per channel. So one RGB pixel = 3 bytes = 24 bits ("24-bit colour").
- The numbers 0–255 are what the display hardware uses to set the intensity of each colour; 0 means "off", 255 means "full".

1.3 2.3 Setup: Imports and Loading an Image

We use **OpenCV** (`cv2`) to load and process images, **NumPy** for array operations, and **Matplotlib** to display them.

[1]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

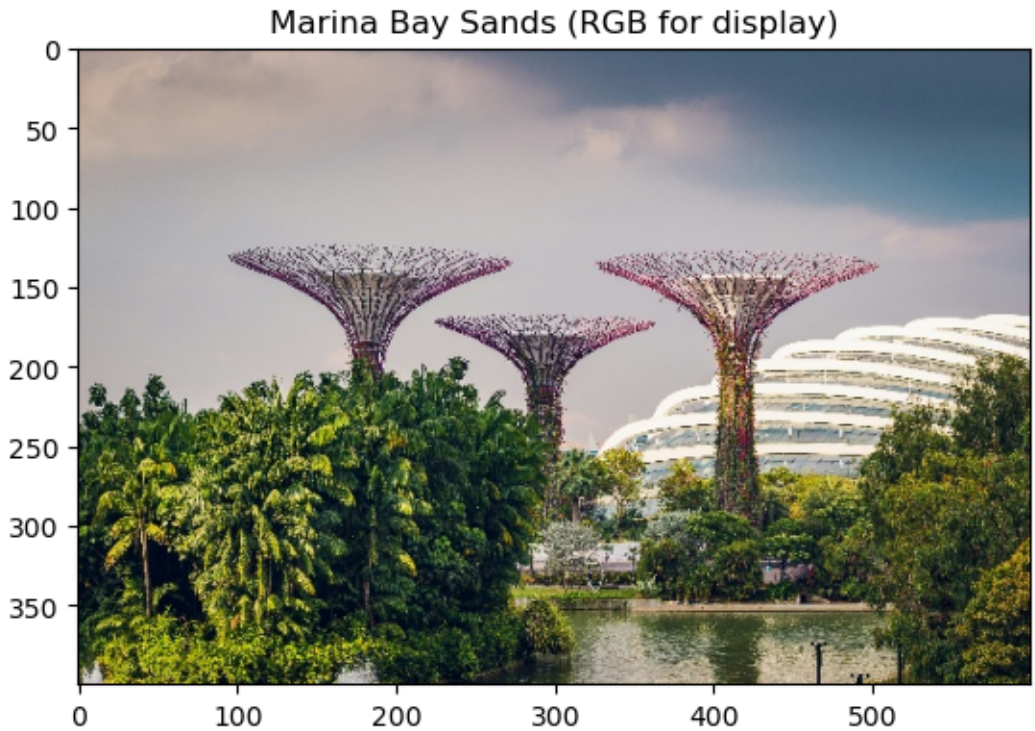
Load a sample image from the `data` folder. We use a path relative to the notebook so it works from the `week-1` directory.

[2]:
```
image_path = 'data/marina_bay_sands.jpg'
img = cv2.imread(image_path)
```

1.3.1 Displaying the image with Matplotlib

Matplotlib expects **RGB** order. OpenCV gives us **BGR**. So we convert before displaying, otherwise colours look wrong (e.g. red and blue swapped).

[3]:
```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.title('Marina Bay Sands (RGB for display)')
plt.show()
```



1.4 2.4 Inspecting the NumPy Array

The image is just a NumPy array. We inspect its **shape**, **dtype**, and **value range**.

[4]:
```
print('Shape:', img.shape)   # (height, width, channels)
print('Dtype:', img.dtype)   # usually uint8 for 8-bit images
print('Min value:', img.min(), '| Max value:', img.max())
```

```
Shape: (400, 600, 3)
Dtype: uint8
Min value: 0 | Max value: 255
```

Interpretation: - `shape` is `(H, W, C)`: height (rows), width (columns), number of channels (3 for BGR). - `dtype` is `uint8`: unsigned 8-bit integer → values 0–255. - Min/max confirm the pixel values lie in that range.

1.4.1 Basic NumPy operations: indexing and slicing

- **Single pixel** (e.g. at row 100, column 200): `img[100, 200]` → array `[B, G, R]`.
- **Region of interest (ROI)**: `img[y1:y2, x1:x2]` gives a rectangular patch.
- **Single channel**: `img[:, :, 0]` is the Blue channel; `img[:, :, 1]` Green; `img[:, :, 2]` Red.

[5]:
```
h, w, c = img.shape
center_y, center_x = h // 2, w // 2
pixel_bgr = img[center_y, center_x]
print(f'Center pixel (B,G,R): {pixel_bgr}')

roi = img[50:150, 100:200]  # region: rows 50-149, columns 100-199
print(f'ROI shape: {roi.shape}')
```

```
Center pixel (B,G,R): [99 85 87]
ROI shape: (100, 100, 3)
```

1.5 2.5 Grayscale and Channel Splitting

Grayscale = single channel. OpenCV can convert BGR → grayscale. We can also extract one channel or combine channels.

[6]:
```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print('Grayscale shape:', gray.shape)  # (H, W) - no channel dimension
print('Grayscale dtype:', gray.dtype, '| Min:', gray.min(), 'Max:', gray.max())
```

```
Grayscale shape: (400, 600)
Grayscale dtype: uint8 | Min: 2 Max: 254
```

[7]:
```
b, g, r = cv2.split(img)
# Build RGB images with only one channel non-zero (so they appear red, green,␣
↪or blue)
red_only_rgb = np.stack([r, np.zeros_like(r), np.zeros_like(r)], axis=-1)
green_only_rgb = np.stack([np.zeros_like(g), g, np.zeros_like(g)], axis=-1)
blue_only_rgb = np.stack([np.zeros_like(b), np.zeros_like(b), b], axis=-1)

fig, axes = plt.subplots(1, 5, figsize=(16, 4))

axes[0].imshow(img_rgb)
axes[0].set_title('RGB')
axes[0].axis('off')

axes[1].imshow(gray, cmap='gray')
axes[1].set_title('Grayscale')
axes[1].axis('off')

axes[2].imshow(red_only_rgb)
axes[2].set_title('Red channel')
axes[2].axis('off')

axes[3].imshow(green_only_rgb)
axes[3].set_title('Green channel')
axes[3].axis('off')
```

```
axes[4].imshow(blue_only_rgb)
axes[4].set_title('Blue channel')
axes[4].axis('off')

plt.tight_layout()
plt.show()
```

1.6 2.6 Storage and Transport Formats

Storage = how we save an image (file on disk, database, cloud). **Transport** = how we send it (API, message queue, network).

- **File formats**: JPEG (lossy), PNG (lossless, can have alpha), BMP (uncompressed). We write to disk with `cv2.imwrite()` and read with `cv2.imread()`.
- **Byte strings**: Many systems (SQL databases, JSON APIs, message queues) cannot store a NumPy array directly. They need **bytes**—a sequence of numbers (0–255) that can be serialized. We encode the image to bytes (e.g. JPEG format), send or store those bytes, then decode back to a NumPy array when needed.
- **Why bytes strings**: A Python `bytes` object is raw binary data. JPEG/PNG files contain byte values that are *not* valid UTF-8, so calling `.decode('utf-8')` raises a `UnicodeDecodeError`. You cannot store raw image bytes in a SQL `TEXT` column or a JSON string directly.
- **Base64**: Encodes binary data into printable ASCII characters (A–Z, a–z, 0–9, `+`, `/`, `=`), producing a plain Python `str`. That string is safe to store in a SQL `TEXT` column, a JSON field, an HTML `data:` URL, or any text-based protocol. Trade-off: ~33 % larger than raw bytes.

Is information lost during encode/decode? Yes—JPEG is **lossy**. Some detail is discarded for smaller file size; the decoded image may show slight compression artifacts. Use PNG (`.png`) with `imencode` for a **lossless** round-trip if you need pixel-perfect reconstruction.

[8]:
```
# Encode: NumPy array → JPEG bytes (what we'd store in a DB or send over a␣
↪network)
success, encoded_buf = cv2.imencode('.jpg', img)
img_as_bytes = encoded_buf.tobytes()   # Python bytes object

print(f'Type   : {type(img_as_bytes)}')
print(f'Length : {len(img_as_bytes):,} bytes')
```

```
print(f'First 40 bytes : {img_as_bytes[:40]}')
print()

# These are raw binary values - NOT a valid UTF-8 string:
try:
    img_as_bytes.decode('utf-8')
except UnicodeDecodeError as e:
    print(f'Cannot decode as UTF-8: {e}')
```

```
Type   : <class 'bytes'>
Length : 140,066 bytes
First 40 bytes : b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00\x00\x01\x00\x01\
x00\x00\xff\xdb\x00C\x00\x02\x01\x01\x01\x01\x01\x02\x01\x01\x01\x02\x02\x02\x02
\x02'

Cannot decode as UTF-8: 'utf-8' codec can't decode byte 0xff in position 0:
invalid start byte
```

[9]:
```
import base64

# Base64 encode: bytes → str  (safe for SQL TEXT, JSON, HTML data URLs, etc.)
b64_string = base64.b64encode(img_as_bytes).decode('utf-8')

print(f'Raw bytes length  : {len(img_as_bytes):,} bytes')
print(f'Base64 str length : {len(b64_string):,} characters  (~33 % more than␣
 ↪raw bytes)')
print()
print(f'First 80 chars : {b64_string[:80]}')
print(f'Last  20 chars : ...{b64_string[-20:]}')
```

```
Raw bytes length  : 140,066 bytes
Base64 str length : 186,756 characters  (~33 % more than raw bytes)

First 80 chars :
/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAIBAQEBAQIBAQECAgICAgQDAgICAgUEBAMEBgUGBgYFBgYG
Last  20 chars : …7tr8zm+qzm7uSXzP/9k=
```

[10]:
```
# --- Decode path A: raw bytes → image ---
nparr = np.frombuffer(img_as_bytes, dtype=np.uint8)
img_decoded = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

# --- Decode path B: base64 string → bytes → image ---
img_bytes_from_b64 = base64.b64decode(b64_string)
nparr_b64 = np.frombuffer(img_bytes_from_b64, dtype=np.uint8)
img_from_b64 = cv2.imdecode(nparr_b64, cv2.IMREAD_COLOR)

# All three should look visually identical (JPEG lossy compression was applied␣
 ↪in imencode)
```

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
axes[0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
axes[0].set_title('Original (NumPy array)')
axes[0].axis('off')
axes[1].imshow(cv2.cvtColor(img_decoded, cv2.COLOR_BGR2RGB))
axes[1].set_title('Decoded from raw bytes\n(np.frombuffer + imdecode)')
axes[1].axis('off')
axes[2].imshow(cv2.cvtColor(img_from_b64, cv2.COLOR_BGR2RGB))
axes[2].set_title('Decoded from base64 string\n(b64decode + imdecode)')
axes[2].axis('off')
plt.tight_layout()
plt.show()
```

1.7 2.7 Application: Colour-based masking with HSV

Why use another colour scale? In **RGB**, a single "colour" (e.g. red) is a mix of R, G, B, so selecting "all red pixels" is awkward. In **HSV** (Hue, Saturation, Value), **Hue** is the colour angle (e.g. red, blue), **Saturation** is how vivid it is, **Value** is brightness. That makes it easier to isolate regions by colour.

Simple application: Isolate bluish tones (e.g. sky or water) using a range in Hue. We convert to HSV, define lower/upper bounds, and use `cv2.inRange()` to get a mask, then show the masked result.

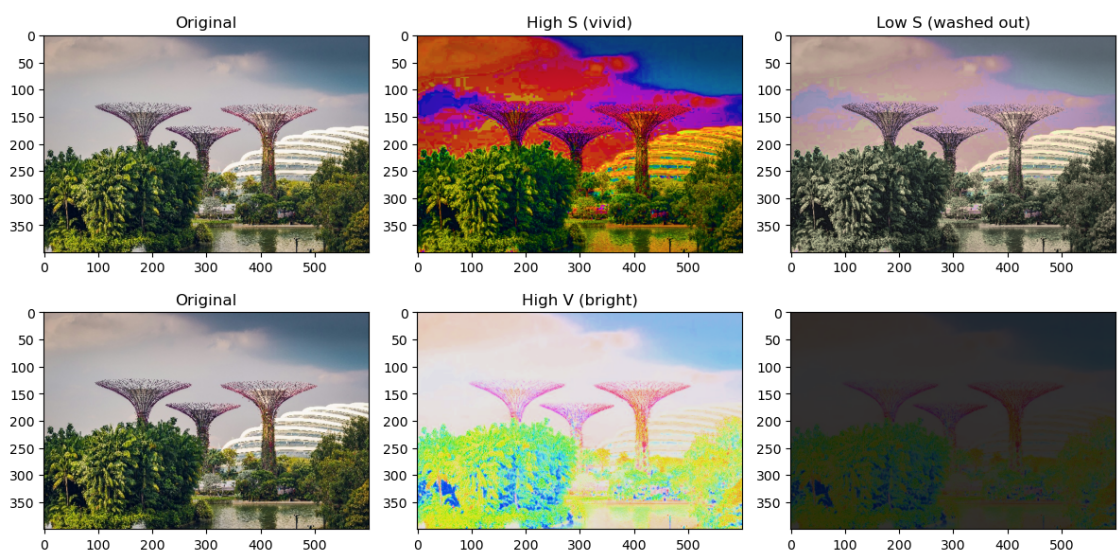
1.7.1 Understanding S (Saturation) and V (Value)

Before masking, let's see what **S** and **V** do: - **Saturation (S)**: How vivid the colour is. High S = vivid, pure colour. Low S = grayish, washed out. - **Value (V)**: Brightness. High V = bright. Low V = dark.

Below we modify S or V in the HSV image and convert back to RGB to visualize the effect.

[11]:
```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Saturation: high S = vivid colour, low S = grayish
hsv_high_s = hsv.copy()
```

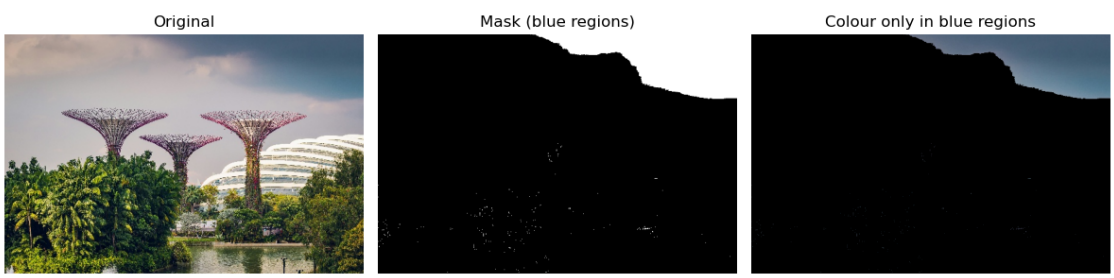
```
hsv_high_s[:, :, 1] = 230  # S channel
hsv_low_s = hsv.copy()
hsv_low_s[:, :, 1] = 50

# Value: high V = bright, low V = dark
hsv_high_v = hsv.copy()
hsv_high_v[:, :, 2] = 230  # V channel
hsv_low_v = hsv.copy()
hsv_low_v[:, :, 2] = 50

def hsv_to_rgb_display(hsv_img):
    bgr = cv2.cvtColor(hsv_img, cv2.COLOR_HSV2BGR)
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)

fig, axes = plt.subplots(2, 3, figsize=(12, 6))
axes[0, 0].imshow(img_rgb)
axes[0, 0].set_title('Original')
axes[0, 1].imshow(hsv_to_rgb_display(hsv_high_s))
axes[0, 1].set_title('High S (vivid)')
axes[0, 2].imshow(hsv_to_rgb_display(hsv_low_s))
axes[0, 2].set_title('Low S (washed out)')
axes[1, 0].imshow(img_rgb)
axes[1, 0].set_title('Original')
axes[1, 1].imshow(hsv_to_rgb_display(hsv_high_v))
axes[1, 1].set_title('High V (bright)')
axes[1, 2].imshow(hsv_to_rgb_display(hsv_low_v))
plt.tight_layout()
plt.show()
```



1.7.2 Colour-based masking with cv2.inRange

Now we use HSV to isolate blue regions. In OpenCV: H is 0–179, S and V are 0–255. Blue hues are around 100–130.

[12]:
```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# In OpenCV: H is 0--179, S and V are 0--255. Blue hues are around 100--130.
lower_blue = np.array([100, 50, 50])
upper_blue = np.array([130, 255, 255])
mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)

# Show original, mask, and masked image (colour only where mask is 255)
masked_rgb = img_rgb.copy()
masked_rgb[mask_blue == 0] = 0  # black out where mask is 0

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(img_rgb)
axes[0].set_title('Original')
axes[0].axis('off')
axes[1].imshow(mask_blue, cmap='gray')
axes[1].set_title('Mask (blue regions)')
axes[1].axis('off')
axes[2].imshow(masked_rgb)
axes[2].set_title('Colour only in blue regions')
axes[2].axis('off')
plt.tight_layout()
plt.show()
```



1.8 2.8 In class: Different ways to darken and brighten

We change brightness by changing pixel values. All methods must keep values in **0–255** (e.g. with `np.clip`) and convert back to `uint8`.

Method 1 — Add/subtract a constant: - Brighter: add a positive number (e.g. +40). - Darker: subtract a positive number (e.g. -40).

Method 2 — Multiply by a factor: - Brighter: multiply by > 1 (e.g. 1.3). - Darker: multiply by < 1 (e.g. 0.6).

Method 3 — Scale then offset: Combine: e.g. `(img * factor) + offset`, then clip. Gives more control (e.g. brighten midtones without blowing out highlights).

[13]:
```
# Work in float to avoid overflow, then clip and convert back to uint8
img_f = img.astype(np.float32)

# Method 1: Add/subtract constant
brighter_add = np.clip(img_f + 50, 0, 255).astype(np.uint8)
darker_add = np.clip(img_f - 50, 0, 255).astype(np.uint8)

# Method 2: Multiply by factor
brighter_mul = np.clip(img_f * 1.35, 0, 255).astype(np.uint8)
darker_mul = np.clip(img_f * 0.6, 0, 255).astype(np.uint8)

# Method 3: Scale + offset (e.g. gentle brighten)
brighter_scale = np.clip(img_f * 1.1 + 20, 0, 255).astype(np.uint8)

# Display: convert BGR to RGB for matplotlib
def bgr_to_rgb(bgr):
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)

fig, axes = plt.subplots(2, 3, figsize=(12, 8))
axes[0, 0].imshow(bgr_to_rgb(img))
axes[0, 0].set_title('Original')
axes[0, 0].axis('off')
axes[0, 1].imshow(bgr_to_rgb(brighter_add))
axes[0, 1].set_title('Brighter (+50)')
axes[0, 1].axis('off')
axes[0, 2].imshow(bgr_to_rgb(darker_add))
axes[0, 2].set_title('Darker (-50)')
axes[0, 2].axis('off')
axes[1, 0].imshow(bgr_to_rgb(brighter_mul))
axes[1, 0].set_title('Brighter (×1.35)')
axes[1, 0].axis('off')
axes[1, 1].imshow(bgr_to_rgb(darker_mul))
axes[1, 1].set_title('Darker (×0.6)')
axes[1, 1].axis('off')
axes[1, 2].imshow(bgr_to_rgb(brighter_scale))
axes[1, 2].set_title('Brighter (×1.1 + 20)')
axes[1, 2].axis('off')
plt.tight_layout()
plt.show()
```

1.9 2.9 Summary

- An image in memory is a **NumPy array** (e.g. shape `(H, W, 3)` for BGR, dtype `uint8`).
- **Channels**: BGR in OpenCV; convert to RGB for Matplotlib. Grayscale = one channel.
- **8-bit** per channel → values **0–255**. Use **shape**, **dtype**, and **slicing** to inspect and manipulate pixels.
- **HSV** is useful for colour-based masking (e.g. `cv2.inRange` on Hue).
- **Brightness** can be changed by add/subtract, multiply, or scale+offset; always clip to 0–255 and use `astype(np.uint8)`.
- Classical CV + ML/DL are complementary; we often use both in real applications.